Synthetic Chest X-Ray Generation and Dataset Augmentation Using Denoising Diffusion Probabilistic Models

Alexander Bowari
alexander.bowari@icloud.com

Background

According to the National Institutes of Health and the National Library of Medicine, "Chest X-ray exams are one of the most frequent and cost-effective medical imaging examinations available." To address the lack of large annotated datasets, the NIH released the Chest X-Ray Dataset containing 112,120 images from 30,805 patients. Although this dataset marked a major step forward and was an extremely significant contribution to the field of medicinal programming and analysis, limitations still remain, particularly in instances like the availability of diverse and balanced training data for machine learning applications. Rare pathologies are often underrepresented, and collecting new labeled medical images is both expensive and time-consuming. To explore a possible solution, I developed a diffusion model capable of generating synthetic chest X-ray images. This project delves into how artificial intelligence generated medical images could help supplement existing datasets and support future improvements in automated disease detection and medical image analysis—complete with explanations of the stochastic processes and other math to make this algorithm possible. This builds upon already renowned research from Jonathan Ho, Ajay Jain, and Pieter Abbeel at the University of California at Berkeley in their paper "Denoising Diffusion Probabilistic Models," which established the framework for future diffusion models. If you would like to receive a copy of the working program of the chest x-ray diffusion model, please request it in my email.

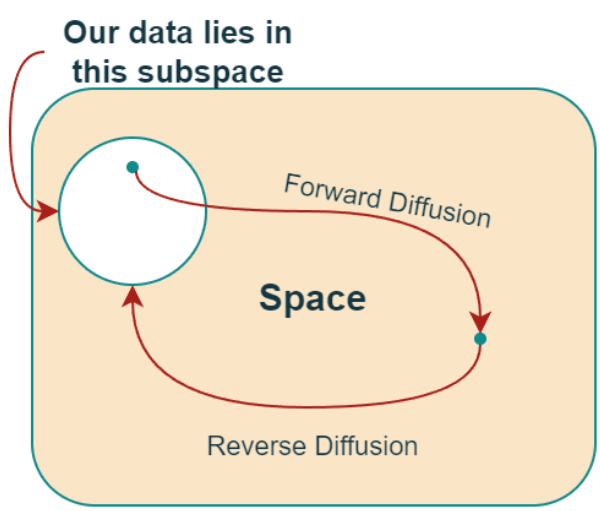
1. Introduction to Diffusion Models

Diffusion models are a class of AI generative models that learn to synthesize data by reversing a gradual noising process. These models consist of two coupled Markov chains: a forward process that destroys structure, and a reverse process that reconstructs it. For an image, this means starting with pure, random gaussian noise, and training a model to remove it. For the forward process, we take an original image and iteratively sample a pixel of gaussian noise and slowly destroy the image until it is pure noise. For the reverse, we train a model to remove gaussian noise iteratively until an image is revealed.

Another way a diffusion model is typically comprehended is by personifying it as navigating through latent space. Think of the space of all possible images as a high-dimensional manifold:

$$X \subset \mathbb{R}^d$$

Real images occupy a thin, structured region of this space, while pure noise lives everywhere else. The forward process is like leaving the manifold: starting at x at timestep 0 on the data manifold, each diffusion step nudges the point off it.



Vaibhav Singh, "An In-Depth Guide to Denoising Diffusion Probabilistic Models DDPM." 2023.

The backward process is the opposite—going back to that space iteratively, starting at the final timestep and making its way back to zero, except periodically taking a step in a distinct direction at random.

Another way diffusion models are typically depicted is in brownian motion. The discrete Markov chain can be written in continuous time as a stochastic differential equation. This then reveals the Brownian motion structure explicitly for the forward process:

$$dx = -\frac{1}{2}\beta(t)xdt + \beta(t)dW$$

Here W is a standard Wiener process—Brownian motion—and β(t) is the continuous noise schedule. This has two terms: The first drift term is a deterministic pull toward the origin, exponentially shrinking the signal. Then the second diffusion term, a random Brownian kick in a uniformly random direction.

$$dx = [-\frac{1}{2}\beta(t)x - \beta(t)\nabla x log p_t(x)]dt + \beta(t)dW$$

With both of these terms, the particle drifts inward towards x_0 while randomly moving outward at each instant. This represents Ornstein-Uhlenbeck dynamics—the stationary distribution is exactly a regular normal distribution *N(0, I)*. These equations similarly will come to resemble our own iterative forward and backward processes, in which we will both derive.

Our model will be trained to generate five different conditions along with its normal or healthy state:

1. Cardiomegaly
2. Edema
3. Pneumonia
4. Pleural Effusion
5. Pneumothorax

Each condition represents a clinically significant pathology commonly identified distinctly in chest radiographs, spanning cardiac, pulmonary, and pleural abnormalities. Cardiomegaly depicts an abnormal enlargement of the cardiac silhouette, while edema refers to fluid accumulation within the pulmonary interstitium and alveoli. Pneumonia manifests as consolidative or interstitial opacities resulting from parenchymal infection, pleural effusion describes the pathological accumulation of fluid within the pleural space, and pneumothorax is characterized by the presence of air between the visceral and parietal pleura. Together these conditions, alongside the healthy baseline, constitute a diverse and diagnostically challenging set of classes that reflect the distributional complexity encountered in

real-world clinical chest imaging datasets, to which the model will learn to encompass through the iterative backward process.

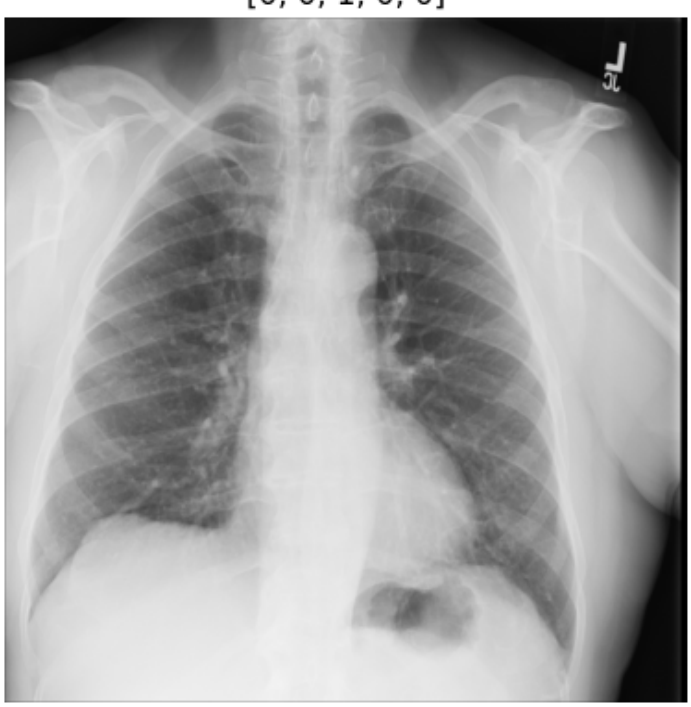
2. Normalizing and Embedding the Data

The first step we can take in tackling this problem is normalizing our data. For programmable simplicity, we can choose to represent the conditions as a multi-hot vector in the order of when we initially defined the diseases. If a disease is present, a 1 will be represented in its space, otherwise a 0 will be represented. For instance, if a chest x-ray happens to have both cardiomegaly and pneumonia, the multi-hot vector would be denoted as [1, 0, 1, 0, 0]. Mathematically, we denote this as such:

$$y(i) = [y_{card}, y_{edem}, y_{pneu}, y_{peff}, y_{ptx}]\top \in 0, 1$$

Drawing a sample from the dataset, we can view some of the images containing conditions with the multi-hot vector:

[0, 0, 1, 0, 0]



In this case, the patient has pneumonia which can be exhibited in the increased whiteness in the lungs, indicating fluid-filled alveoli.

To ensure label quality and exclude ambiguous or out of the ordinary cases, we compute the column-wise label sum across all samples. For a single image, we define its label as:

$$\| \mathbf{y}^{(i)} \| \sum_{k=1}^{5} \mathbf{y}_k^{(i)}$$

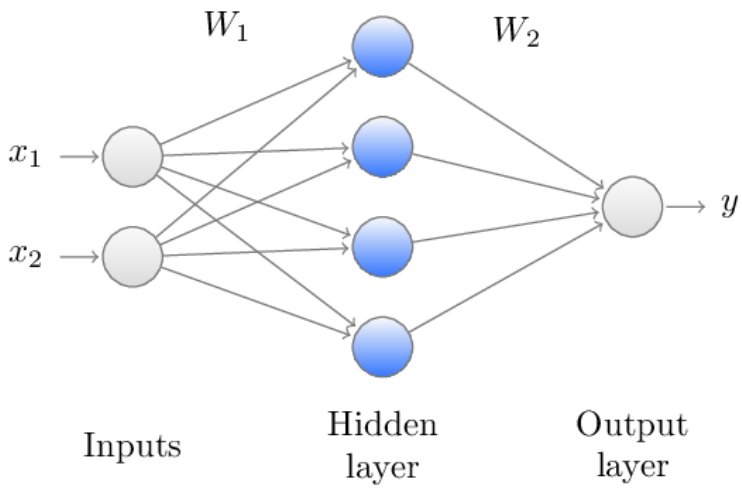
Fundamentally, this means adding up all the values across the column. Then, an image is retained in the training set if and only if it satisfies the following criterion:

$$keep(i) \iff \mathbf{y}^{(i)} \in [0, 1]^5$$

That is, an image is kept if it is classified as normal (having a summation of 0 and classified as the healthy state) or if at least one of its labels belongs to the five known conditions. Images whose label vectors contain values outside this known set—which indicates unlabeled or uncertain pathologies—are discarded. This filtering step ensures the model is never exposed to disease categories without a well-defined conditioning signal, preventing the latent space from being corrupted by ambiguous supervision and preventing from generating unclassified diseases and labeling them as healthy.

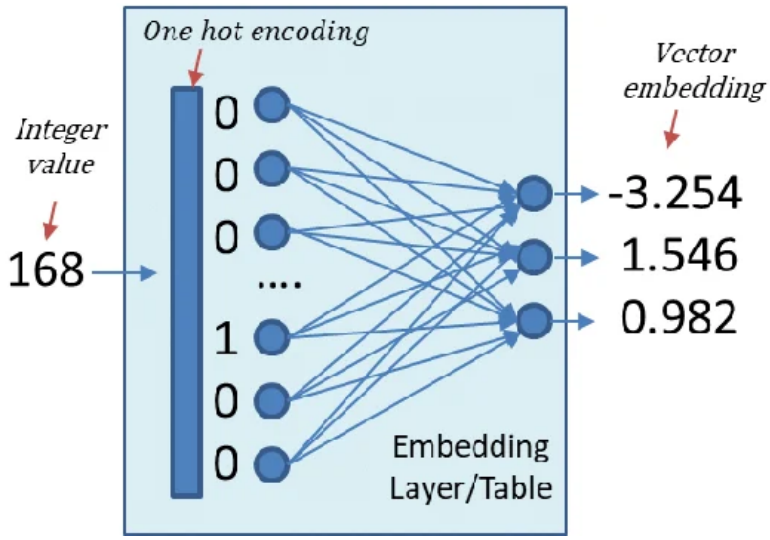
To reduce computational cost while preserving diagnostically relevant spatial features, all chest radiographs are resized from their native resolution of 1024×1024 pixels to a standardized resolution of 256x256 pixels.

Raw multi-hot vectors are discrete and sparse. Therefore, feeding them directly into the diffusion model as conditioning signals would provide insufficient expressive capacity for complex patterns: the model would struggle to learn smooth, continuous relationships between conditions in the latent space. To address this, we project the label vector into a dense, high-dimensional embedding space via a two-layer feed-forward network.



Bogdan Penkovsky, "Day 2. What Do Hidden Layers Do?" 2019.

In this case, we will have 5 inputs—delineated by our five conditions—which feed forward into a first layer of 64 neurons and then onto a second layer of 128 neurons. ReLU activations are placed between each linear transformation to introduce non-linearity, enabling the network to learn complex, non-linear interactions among co-occurring conditions. For instance, the joint presence of Edema and Pleural Effusion may share structural radiographic features that a purely linear projection could not capture, so we use a Rectified Linear Unit to assist in capturing these complex patterns. The second layer acts as our output—returning 128 activations that act as an encoded embedding for the 5 chest conditions.



Manuel Lopez-Martin, "Network intrusion detection with a novel hierarchy of distances between embeddings of hash IP addresses." 2021.

Finally, to finish normalizing the data, each retained image undergoes a two-stage normalization pipeline prior to being passed to the model. First we convert to a tensor; since raw pixel intensities are represented as unsigned 8-bit integers, the "ToTensor" operation maps each pixel to the unit interval:

$$p = p \in [0, 1, \ldots, 255]$$
$$p = \frac{p}{255} \in [0, 1].$$

In our second step we normalize by making the unit interval to be zero-centered. The unit-scaled pixel p is then linearly transformed to a symmetric range using channel-wise mean $\mu = 0.5$ and standard deviation $\sigma = 0.5$:

$$p = \frac{p - \mu}{\sigma} = \frac{p - 0.5}{.5} = 2p - 1 \in [-1, 1].$$

This zero-centering is critical for diffusion model training: since the forward process targets *N(0, I)*, images must be scaled to a range whose statistics are compatible with the Gaussian noise schedule. Centering at zero with unit variance ensures that the signal-to-noise ratio prevents the model from having to implicitly learn a constant offset throughout the denoising process.

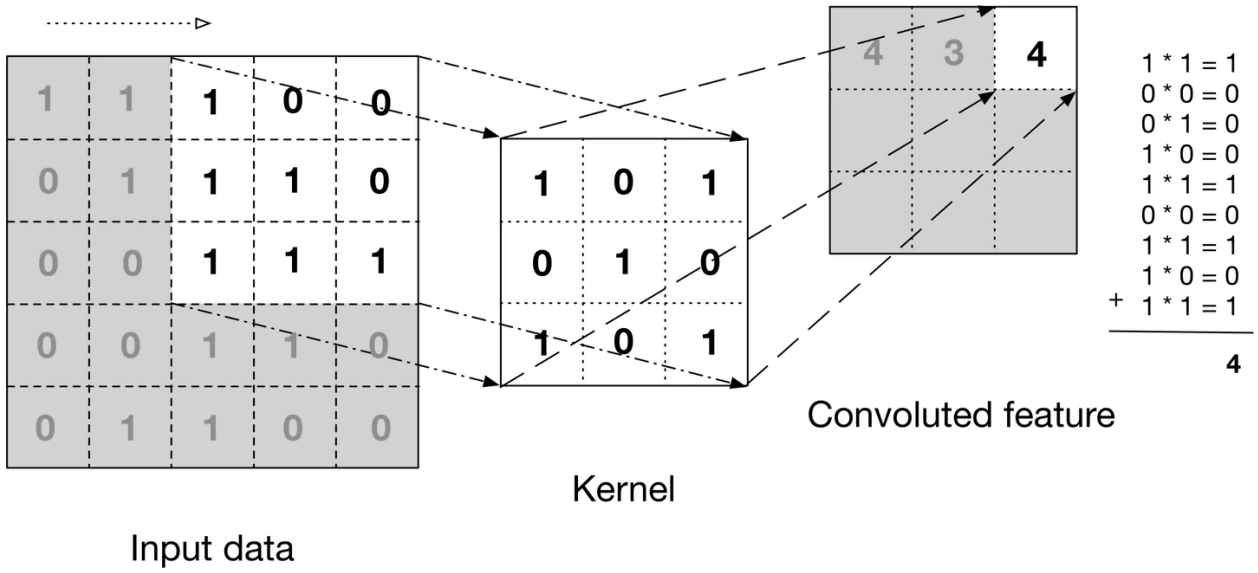
3. Encoding the Images Themselves

To extract meaningful visual features from each chest radiograph, we pass the image through a convolutional encoder that progressively compresses spatial resolution while expanding representational depth. The input to the encoder is a single-channel grayscale image of resolution 128x128, one channel because chest X-rays carry intensity information only, with no color.

A channel in this context can be thought of as a feature map: a 2D grid where every spatial position stores one learned measurement about that location in the image. The input has one such map (raw pixel brightness), but as the image passes through the network, the encoder learns to compute many different measurements simultaneously: edges, textures, shapes, densities, with each pattern stored as its own channel. More channels fundamentally means the network is looking at the image through more learned lenses at once.

Each convolutional layer scans the image using a 3x3 kernel—a small sliding window that looks at a 3x3 square of pixels at every position and computes a weighted sum or a convolution of the values, learning

to detect local patterns such as edges or boundaries and calculated as so below. A ReLU activation follows each convolution, discarding negative responses and introducing the non-linearity the network needs to represent complex visual structure.



Manav, "Convolutional Neural Networks (CNN) in Deep Learning." 2025.

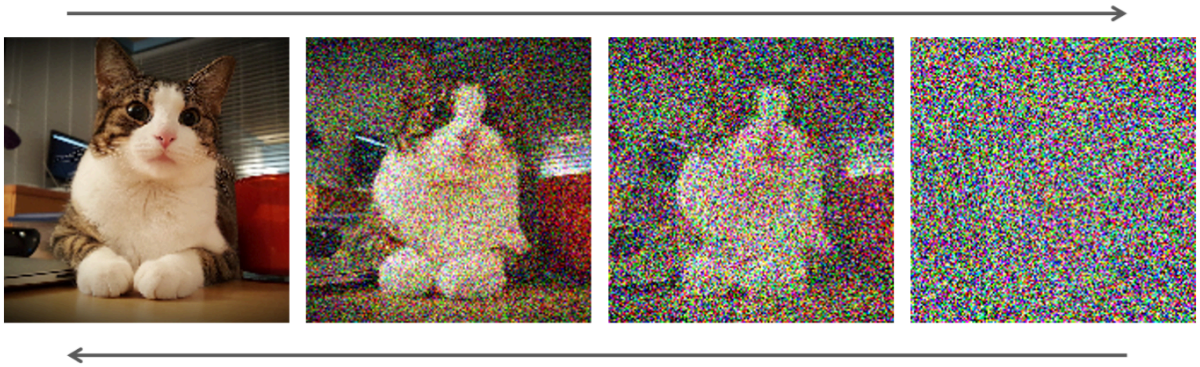
After the first convolutional block, a max pooling (or MaxPool) operation halves the spatial dimensions by taking the maximum activation in each 2x2 region, retaining the strongest detected features while discarding redundant spatial detail. This produces a representation of size 64x64 with 64 channels: the image has been spatially compressed by a factor of two, but is now described by 64 simultaneous feature maps rather than one.

The same process is then applied again, except with different specificities: a convolutional block with a ReLU, followed by max pooling, which halves the spatial resolution once more and doubles the channel depth. The output is a tensor of size 32x32 tensor with 128 channels—turning it into an encoded, embedded, and learned batch of features represented as a 32x32x128 dimensional representation.

4. The Forward Diffusion Process

To first develop a model that goes backwards from pure noise, we need to first train the neural network on an image being corrupted iteratively through the noise. This is done by repeatedly sampling

gaussian noise and adding it to a clear image in continuous steps—eventually completely corrupting it so that it just simply becomes pure gaussian noise.



Arash Vahdat and Karsten Kreis, Improving Diffusion Models as an Alternative To GANs
2022.

We'll call the image we start with x_0 and the final, pure noisy image x_T. To get from x_o to x_T, we'll first demonstrate how this works by simply computing x_0 to x_1. For this, we'll use a conditional distribution—fundametnally, retrieve the probability density of x_1 given x_0:

$$q(x_1|x_0)$$

We produce x_1 by adding an amount of gaussian noise times a variable beta. Moreover, beta is controlled by a linear noise schedule, where beta increases linearly across a timestep T of 1000 starting from beta start—which I defined as $1x10^{-4}$—and beta end—which I defined as $2x10^{-2}$:

$$\beta_t = \beta_{start} + \frac{T-1}{t-1}(\beta_{end} - \beta_{start})$$
$$t = 1, \ldots, 1000$$
$$x_1 = x_0 + \beta\epsilon$$
$$\epsilon \sim N(0, I)$$

The probability distribution of x_1 is essentially just a gaussian centered on x_0 with noise variance beta:

$$q(x_1|x_0) = N(x_0, \beta)$$

The next step is almost exactly the same as x_1, where we compute x_2 by adding x_1 to beta multiplied by epsilon. This again is also just the same thing as $q(x_2|x_1) = N(x_0, beta)$. To go straight from x_0 to x_2 however, we can simply add twice as much noise:

$$x_2 = x_0 + 2\beta\epsilon$$
$$q(x_2|x_0) = N(x_0, 2\beta)$$

We can repeat this step for each timestep t to ultimately achieve a more proper formula:

$$x_t = x_0 + t\beta\epsilon$$
$$q(x_t|x_0) = N(x_0, t\beta)$$

However, since our goal is to get x_t to x_0 to converge to a normal distribution of $N(0,1)$, we have to have the mean converge to zero and the variance to one. To do this, we'll need a coefficient in front of the mean to make it decrease to zero. For this, we'll define a_t as the amount of noise left in the image, with a_t equal 1 minus beta to the power of t. This will turn the equation into the following:

$$a_t = (1 - \beta)^t$$
$$q(x_1|x_0) = \sqrt{a_t}x_0 + (1 - a_t)\epsilon$$
$$q(x_t|x_0) = \sqrt{a_t}x_0 + (1 - a_t)^t\epsilon$$

That bottom equation is our final expression for the entire forward process. This means that the conditional distribution of x_t given x_0 converges to zero and the variance goes to one iteratively. If we choose beta between 0 and 1, then a_t converges to 0 as t approaches infinity:

$$a_t = \prod_{t=1}^{t}(1 - \beta_t)$$

5. The Backwards Diffusion Process - Calculating the Loss of the Network

Now that we have established the whole forward process, our goal is to find the best parameters to remove noise from an image effectively enough to produce a clear, distinguished image. For this, we

need to minimize the log-likelihood of a sample—maximizing this likelihood of generating real images of x_0 will maximize the necessary weights or parameters to do this. For formally, we need to compute:

$$-logp_\Theta(x_0)$$

Let's start with an image x_t and move backwards to an image x_{t-1}—to do this we need to compute the probability distribution of x_{t-1} given x_t:

$$p(x_t|x_{t-1})$$

Since we want to compute the probability distribution of x_t, x_{t-1}, and so on given x_0, we can use Bayes' theorem to simplify. This will decompose the joint probability into individual conditional distributions:

$$q(x_{1...T}|x_0) = q(x_1|x_0)q(x_2|x_1)...q(x_t|x_{t-1}...x_0)$$

$$q(x_{1...T}|x_0) = q(x_1|x_0)q(x_2|x_1)...q(x_T|x_{T-1})$$

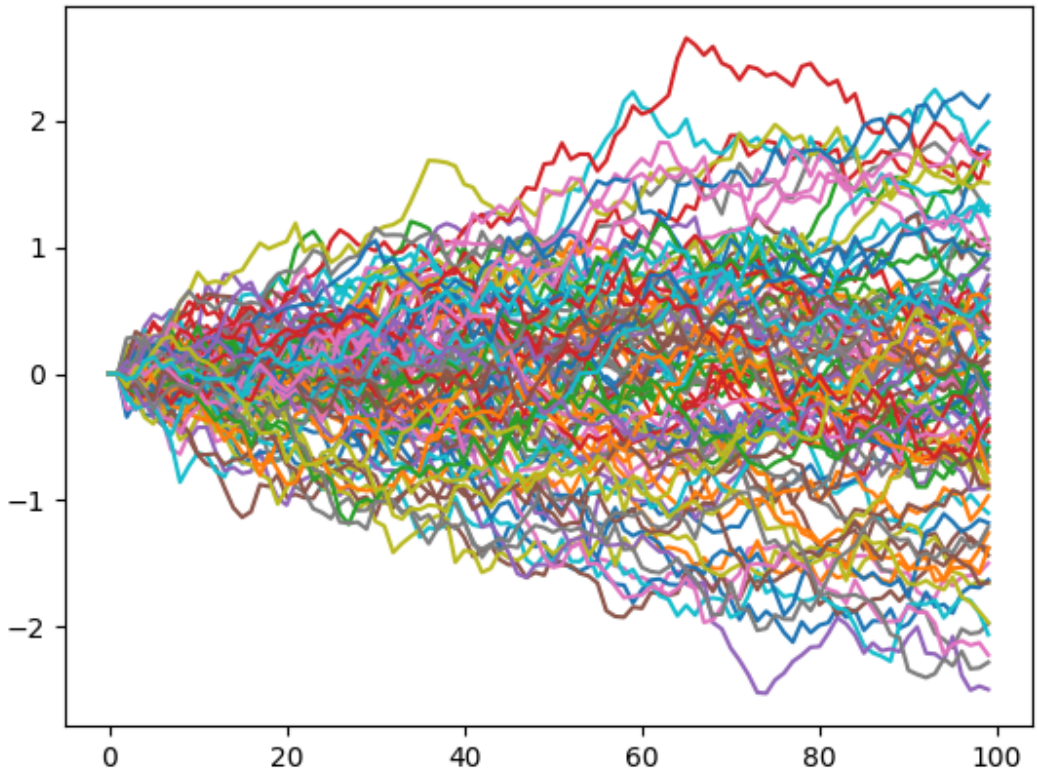
$$q(x_{1:T}|x_0) = q(x_1|x_0)q(x_2|x_1)...q(x_T|x_{T-1})$$

$$q(x_{1:T}|x_0) = \prod_{t=1}^{T} q(x_t|x_{t-1})$$

The complete backwards process without simplification and with the correct parameters can be denoted as such:

$$p_\Theta(x_{1:T}|x_0) = \prod_{t=1}^{T} p_\Theta(x_t|x_{t-1})$$

When dealing with joint probabilities like this one, it's imperative to marginalize the distribution with respect to the other variables. We want the negative log of producing a clear image x_0, so we have to add up all the possible ways the hidden variables can exist. This is similar to marginalization in brownian motion as well:



Valerie Han, "An introduction to Stochastic Calculus." 2019.

We calculate this by taking the integral with respect to all the variables defined in the joint distribution:

$$-log \int p_\Theta(x_{0:T}) dx_{1:T}$$

This integral would give us the probability that our neural network generates sample x_0 by summing all possible paths of generating it. However, in practice this quantity is intractable—it cannot be quantified exactly. To approximate the integral however, we can compute the "evidence lower bound," or ELBO.

To calculate this, we can introduce the forward process by multiplying the density of the forward probability—the conditional probability of producing $x_{1:T}$ given x_0—as a variational posterior:

$$-logp_\Theta(x_0) = \int \frac{p_\Theta(x_{0:T})}{q(x_{1:T}|x_0)} q(x_{1:T}|x_0) dx_{1:T}$$

This is an expectation of the ratio p of q under distribution q. This only reformulates the equation since we are technically multiplying by 1. Now, we can express it like this due to log and expectation rules:

$$-logp_\Theta(x_0) = -log\mathbb{E}_{q(x_{1:T}|x_0)}[\frac{p_\Theta(x_{0:T})}{q(x_{1:T}|x_0)}]$$

From here, we apply the principle of Jensen's Inequality in order to put the log inside the expectation—which relates the value of a convex function of an expectation to the expectation of the function itself. Essentially, for any convex function, the expectation of a log is less than or equal to the log of an expectation:

$$(\phi(\mathbb{E}[X]) \leq \mathbb{E}[\phi(X)]$$

$$\mathbb{E}[log(X)] \leq log(\mathbb{E}[X])$$

$$-logp_\Theta(x_0) \leq -log\mathbb{E}_{q(x_{1:T}|x_0)}[\frac{p_\Theta(x_{0:T})}{q(x_{1:T}|x_0)}]$$

Now we establish the evidence lower bound (ELBO) mentioned earlier. Simply, the ELBO decomposes the intractable integral into a sum of Kullback-Leiber divergences—which are basically measurements of how close the probability distributions of two functions are from each other in probability space—between Gaussians, each of which is cheap to compute and turns training into simple noise prediction. The Kullback-Leiber divergences will be referred to as D_{KL}, and simplifying our equation with the ELBO results in the following expression:

$$-\mathbb{E}_q[D_{KL}(q(x_T|x_0)||p(x_T)) + \sum_{t>1}^{t} D_{KL}(q(x_{t-1}|x_t, x_0)||p_\Theta(x_{t-1}|x_t) - logp_\Theta(x_0|x_1)]$$

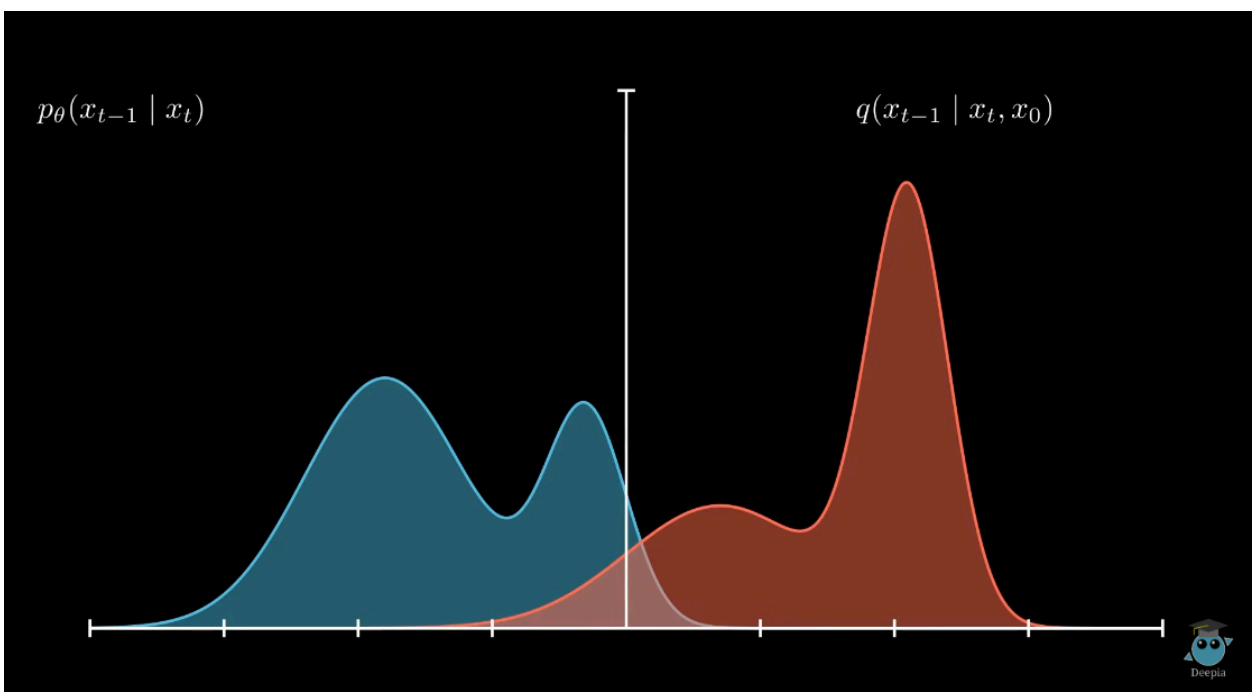
This expression, while very complex, can be simplified when training the model. We can break this equation down by focusing on each of its parts. The first D_{KL} term measuring the difference between the probability distribution of x_T given x_0 with the probability of generating a sample x_T does not depend on theta, or the parameters of our model. Therefore, we can ignore it in optimization:

$$-\mathbb{E}_q[\sum_{t>1}^{t} D_{KL}(q(x_{t-1}|x_t, x_0)||p_\Theta(x_{t-1}|x_t) - logp_\Theta(x_0|x_1)]$$

The next step in simplification is in the last term of the expression—the negative log of the probability distribution with respect to the model's parameters of generating a clean image x_0 given a slightly more noisy one x_1. Since we basically calculate this value multiple times over in the main expression, this number is extremely small and insignificant compared to the other numbers. Therefore, we can ignore it because it makes virtually no difference in optimization:

$$-\mathbb{E}_q[\sum_{t>1}^{t} D_{KL}(q(x_{t-1}|x_t, x_0)||p_\Theta(x_{t-1}|x_t)]$$

That equation above leaves us with the main expression. To break this down simply, this is the sum of Kullback-Leiber divergences between two gaussians for t > 1. The first gaussian is referred to as the true posterior—the true reverse process or "answer" to the reverse process. We compare this to our own model's diffusion step repeatedly and minimize the D_{KL} divergence terms by modifying the model's parameters.



Deep AI, "Diffusion Models, DDPM | Generative AI Animated." 2025.

Essentially, the loss we computed is basically training the neural network's parameters to match the true posterior. Moreover, through algebraic manipulation, we can deduce that the true posterior is actually just $N(\mu_\theta, \beta_\theta)$, or a regular gaussian.

Now, we need a form for the approximate posterior to compare it too—which has to be gaussian as well due to the nature of Markov Chains because the reverse of a gaussian Markov Chain also has to be gaussian. Additionally, for personal preference we defined the variance, sigma, as a constant unchanged by the parameters of the network because of its fairly insignificant contribution to the D_{KL} divergence term. With this we can turn the D_{KL} expression through mathematical manipulation into a simple difference of means between each probability distribution:

$$p_\Theta(x_{t-1}|x_t) = N(\mu_\Theta|\sigma)$$
$$D_{KL}(q(x_{t-1}|x_t, x_0)||p_\Theta(x_{t-1}|x_t)) = \frac{1}{2\sigma^2}||\mu_t - \mu_\Theta||^2$$
$$-\mathbb{E}_q[\sum_{t>1}^{t} D_{KL}(q(x_{t-1}|x_t, x_0)||p_\Theta(x_{t-1}|x_t))] = -\mathbb{E}_q[\frac{1}{2\sigma^2}||\mu_t - \mu_\Theta(x_t, t)||^2]$$

Overall, this is a difference between the predicted mean and the true posterior mean. With this equation, we can simplify even more by deriving the differences in current noise and predicted noise. We need the probability distribution of x_{t-1} given both x_t and x_0. Earlier, we described the forward process as such:

$$q(x_{t-1} \mid x_0) = N(\sqrt{\alpha_{t-1}}x_0, (1-\alpha_{t-1})I)$$
$$q(x_t \mid x_{t-1}) = N(\sqrt{\alpha_t}x_{t-1}, (1-\alpha_t)I)$$

Using Baye's theorem again—which states that joint conditional probabilities are equal to the product of those probabilities—we can state:

$$q(x_{t-1} \mid xt, x0) \propto q(x_t \mid x_{t-1}) \cdot q(x_{t-1} \mid x_0)$$

We can then individually plug them into our derived loss formula. For the first gaussian that computes the probability of x_t given x_{t-1}, we can plug in to get:

$$q(x_t \mid x_{t-1}) = \mathbb{E}(\frac{-1}{2(1-\alpha_t)} \parallel x_t - \sqrt{\alpha_t}x_{t-1} \parallel^2)$$

For the second gaussian that computes the probability of x_{t-1} given x_0, we can plug in to get:

$$q(x_{t-1} \mid x_{t0}) = \mathbb{E}(\frac{-1}{2(1-\alpha_{t-1})} \parallel x_{t-1} - \sqrt{\alpha_{t-1}}x_0 \parallel^2)$$

Now we simplify the exponents and multiply the gaussians. At this step, we can use incredibly extensive algebra and probabilistic knowledge to compute the mean. For simplistic purposes, we'll simplify straight to the answer that calculates this:

$$\mu_q = \frac{\sqrt{a_t}(1-a_{t-1})}{1-a_t}x_t + \frac{\sqrt{a_{t-1}}(1-a_t)}{1-a_t}x_0$$

Fundamentally, this is the exact posterior mean—it depends on both x_t and x_0. Now from the forward, process, we can find x_0 to plug into that equation and simplify:

$$x_t = \sqrt{\alpha_t}x_0 + \sqrt{1-\alpha_t}\epsilon$$
$$x_0 = \frac{1}{\sqrt{\alpha_t}}(x_t - \sqrt{1-\alpha_t}\epsilon)$$

From here we run into a problem: We cannot fully compute x_0 because we don't know the amount of noise at that specific timestep. To work around this, we predict it indirectly via the predicted noise given x_t and timestep t:

$$x_0 = \frac{1}{\sqrt{\alpha_t}}(x_t - \sqrt{1-\alpha_t}\epsilon_\Theta(x_t, t))$$

Plugging this into the formula, we are left with a fairly large expression for the mean of the true posterior. Because our goal originally was to find a simpler expression by representing the equations' differences in noise, since those values are tractable, we can work around having to find the giant equations for μ_t—which I called μ_q in the equation—and μ_θ respectively. Since we know both terms have the same coefficient next to x_t and the same coefficient next to x_0, after algebra this becomes:

$$\mu_q - \mu_\theta \propto (\epsilon - \epsilon_\theta(x_t, t))$$

Therefore, the loss of the network can be rewritten simply as:

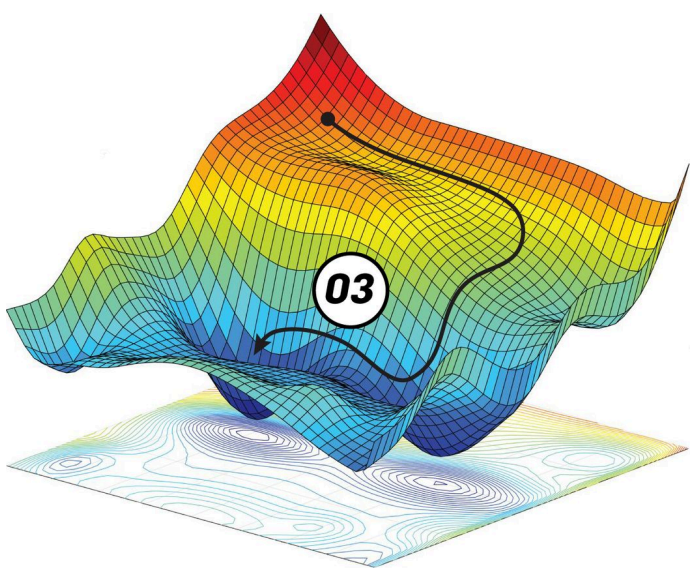
$$\mathbb{E}[\| \epsilon - \epsilon_\theta(x_t, t) \|^2]$$

$$Loss = \mathbb{E}_{t,x_0,\epsilon}[\| \epsilon - \epsilon_\theta(x_t, t) \|^2]$$

This is the final derived loss we will use to optimize the parameters of the network by minimizing this value.

6. Optimizing the Network - Gradient Descent

The process that many large language models, high performing AI models, and several other machine learning algorithms learn is through gradient descent. Gradient descent is the optimization algorithm responsible for minimizing the loss function; conceptually, it operates by computing the partial derivative, or gradient, of the loss function with respect to each parameter in the network. This gradient indicates both the direction and rate of steepest increase in error. To reduce error, the model updates each parameter in the opposite direction of its gradient. Mathematically, this gradient is computed by taking the derivative of the loss function, also referred to as the cost function, with respect to the weights or parameters.



Ahmed Fawzy Gad, "Implementing Gradient Descent in Python." 2019.

The weights are then updated iteratively by subtracting the old weight by the multiplication of the gradient with a learning rate n, often consisting of a small decimal. Functionally, this process looks like picking a spot on the loss function and taking small steps down it to reach the bottom.

$$\Theta_{new} = \Theta_{old} - n\nabla L$$
$$\Theta_{new} = \Theta_{old} - n\frac{\partial L}{\partial \Theta}$$

To do this, we have to compute the partial derivative of the loss function with respect to the network's parameters, theta. To calculate this we'll first start with one sample of the loss and take the derivative:

$$L = \| \epsilon - \epsilon_\theta(x_t, t) \|^2$$
$$\nabla_\Theta L = 2(\epsilon_\Theta - \epsilon)\nabla\epsilon_\Theta(x_t, t)$$

Therefore, the total or real gradient for all samples at a randomly chosen timestep can be written as:

$$\nabla_\Theta L = \mathbb{E}[2(\epsilon_\theta - \epsilon)\nabla_\theta\epsilon_\theta]$$

This loss is then multiplied by a chosen learning rate—which we defined on the model as $1x10^{-4}$—and then subtracted by the old parameters to equate new ones. We define the loss as negative because the gradient or slope actually points upwards, so multiplying it by negative one will consequently go down on the graph. Running this iteratively updates the parameters and increases model accuracy, resulting in a clearer, more distinguished final image.

7. Implementing the Conditions

Because we want to generate an image based on a prompted disease, the model we made is conditional. Instead of generating an image blindly, it is also given a disease condition vector that tells it what findings should be present in the final image. In this project, that condition is represented as a 5-dimensional multi-hot vector, and then embedded earlier which better reflects real chest X-ray data by defining feature maps through high dimensionality. That embedding layer is trained jointly with the diffusion model—during training, it learns associations between disease labels and the visual structures that tend to appear in the corresponding chest X-rays, represented by a 128 dimensioned vector. For instance, a pleural effusion label often corresponds to fluid opacity near the lung bases, so the

embedded structure embeds that information. Thus, instead of the model predicting x_t given t, it has to predict x_t given t and c.

$$\epsilon_\theta = \epsilon(x_t, t)$$
$$\downarrow$$
$$\epsilon_\theta = \epsilon(x_t, t, c)$$

Diffusion generation starts from random noise and gradually transforms it into an image. Without conditioning, that denoising path is unconstrained. The model just drifts toward any image it thinks is plausible. However, once a disease condition is provided, that condition biases the entire denoising process. Instead of moving toward just any chest X-ray, the model is nudged toward the subset of chest X-rays that match the requested disease pattern, or a cluster of high dimensional values in image space.

The training loss stays almost the same as standard diffusion, except the model is now conditioned based on our five inputted states or the normal state. Therefore, the loss now is:

$$L = \mathbb{E}_{x_0,\epsilon,t,c}[\| \epsilon - \epsilon_\theta(x_t, t, c) \|^2]$$

The model is now conditioned to predict the noise x_t given the current timestep and the embedded conditions.

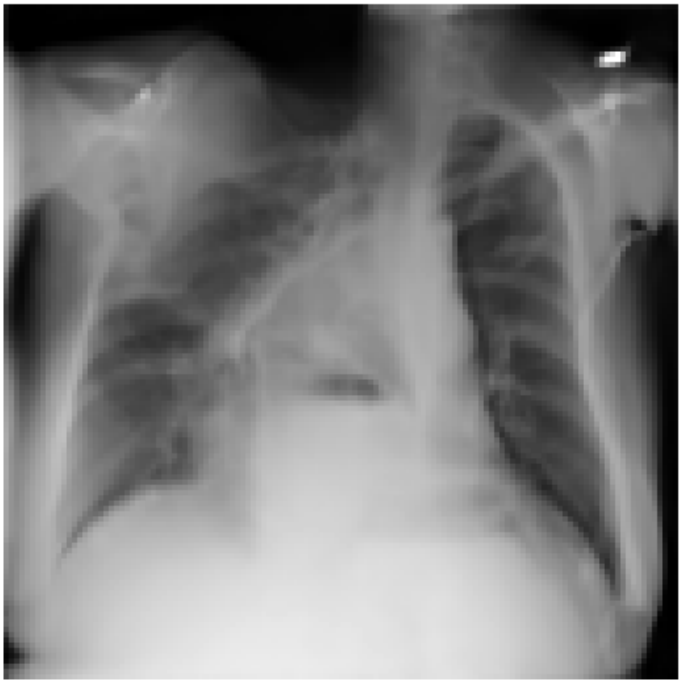
8. Results from the Diffusion Model

To assess the generative capability of the trained diffusion model, we visually inspected samples synthesized conditioned on each of the six diagnostic classes: the five target pathologies and the normal state. As we stated earlier, the five diseases we defined were cardiomegaly, edema, pneumonia, pleural effusion, and pneumothorax.

Overall, the model demonstrated a meaningful capacity to generate chest radiographs that are visually plausible and structurally coherent, capturing the broad anatomical layout expected of a frontal chest X-ray including the lung fields, cardiac silhouette, and ribcage. However, generation quality varied considerably across diagnostic classes, reflecting the uneven class distribution present in the training data and the inherent visual subtlety of certain pathologies.

The first sample we can look at is when the model is trained at ten epochs, at a batch size of 16, with each resolution at 128 pixels by 128 pixels:

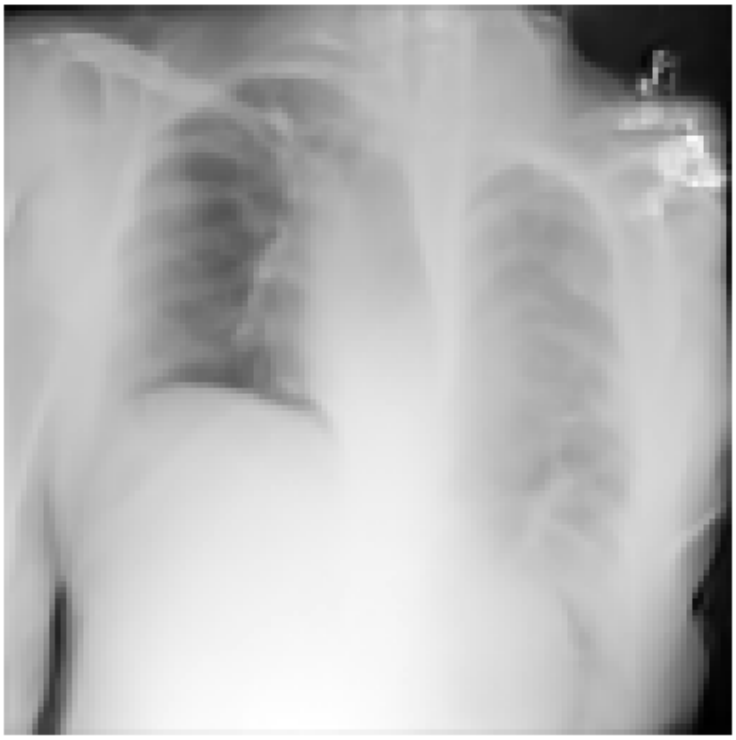
Condition: [0, 0, 0, 0, 0]



This image generated fairly well—lung definition and bone structure is generally consistent with regular radiology scans. As visible from the multi-hot vector, this image has no conditions and is in a healthy state.

Now, we can take a look at some of the synthetic x-rays with diseases, much like this one:

Condition: [0, 0, 1, 0, 0]



From the multi-hot vector, this synthetic chest x-ray appears to be a patient diagnosed with pneumonia. While not perfect, this x-ray was also generated pretty well in only 10 epochs—the overall structure of the lungs and bones seem to be generally in the right places. Overall, pneumonia samples exhibited consistent manners across generated examples. The model appeared more confident in rendering this condition compared to others, potentially owing to the high intra-class variability of pneumonia's radiographic presentation, which can range from subtle ground-glass opacity to dense lobar consolidation. As in the above photo, pneumonia is typically revealed through white patches, known as consolidations, in the lungs where air sacs are filled with fluid or pus.

The rest of the batches of diseases struggled sometimes and excelled at others. Pneumothorax proved to be among the most challenging conditions for the model to render convincingly. The defining radiographic feature—a visible pleural line with absent lung markings peripheral to it—is a highly localized and structurally subtle finding that requires precise spatial reasoning. Generated samples showed inconsistent evidence of this feature, with many samples failing to produce a clearly identifiable pleural boundary or even the structure of a normal chest at all. This likely reflects both the relative scarcity of pneumothorax cases in the training data and the difficulty of encoding spatially precise,

fine-grained pathological features through the conditioning embedding alone. In this case, the model struggled at generating an image of a chest x-ray containing pneumothorax:

Condition: [0, 0, 0, 0, 1]

Edema, pleural effusion, and cardiomegaly generated well too. The variation in generation quality across classes can be attributed to three interacting factors. First, class imbalance in the training set means that the model receives substantially more gradient signal from well-represented conditions, leading to stronger generative fidelity for those classes. Second, pathologies with globally salient and spatially consistent features, such as Cardiomegaly and Pleural Effusion, are inherently more learnable from a conditioning standpoint, as the label embedding can reliably steer the denoising trajectory toward distinctive morphological regions. Third, conditions characterized by subtle, diffuse, or spatially variable findings (like edema and pneumothorax) impose a heavier burden on the condition embedding network, which must compress all discriminative information into a fixed 128x128 dimensional vector shared with the image representation.

9. Conclusion

To conclude, this project and study set out to answer the question of whether a diffusion model is capable of learning how to generate realistic chest X-rays that correspond to a specific disease or label. After building the entire pipeline from scratch—from filtering and embedding the data, encoding the images through a convolutional network, deriving the full forward and backward diffusion processes mathematically, and optimizing the network through gradient descent—the answer for the most part appears to be yes.

The model was capable of generating visually coherent chest radiographs in as little as ten training epochs. For conditions like Pneumonia and Cardiomegaly, the results were genuinely encouraging—the generated images reflected real, recognizable radiographic patterns, with consolidations appearing in the lungs for pneumonia and an enlarged cardiac silhouette showing up for cardiomegaly. Other conditions, particularly Pneumothorax, were harder to solve. The pleural boundary that defines a pneumothorax is subtle and spatially precise, and compressing that kind of fine-grained detail into a 128-dimensional embedding vector is simply a hard problem, one that more sophisticated conditioning architectures would need to address.

The bigger picture however is what this kind of model could actually mean for medicine. One of the most persistent problems in clinical AI is that rare diseases don't have enough labeled data to train good classifiers on. Getting new annotated chest X-rays is slow, expensive, and heavily dependent on radiologist time. A model that can generate synthetic images conditioned on a specific disease state is a real, practical step toward solving that—giving future classifiers more examples of the hard cases to learn from, and potentially improving diagnostic performance exactly where it matters most.

This project built directly on the framework introduced by Ho, Jain, and Abbeel at UC Berkeley in their Denoising Diffusion Probabilistic Models paper, and it showed that those ideas translate meaningfully into conditional medical image generation. There's a lot of room to go further—cross-attention conditioning, several more training epochs on a more powerful computer, higher resolution outputs, and proper quantitative evaluation with metrics would all be natural next steps. But as a proof of concept, this demonstrated something extensively useful: with the right stochastic processes, the right architecture, and a well-defined training objective, a model can learn to navigate from pure Gaussian noise all the way to a synthetic chest X-ray that carries the visual signature of a real disease.

This project was built on Kaggle utilizing the following imports: Numpy, Pandas, Glob, MatPlotLib, and PyTorch. Special thanks to teachers at Jesuit Dallas for assisting in the mathematical solving process and explanation, and as well as Sebastian Alvarez for helping with this project.

NIH Chest X-Ray Dataset:

Kufel J, Bielówka M, Rojek M, Mitręga A, Lewandowski P, Cebula M, Krawczyk D, Bielówka M, Kondoł D, Bargieł-Łączek K, Paszkiewicz I, Czogalik Ł, Kaczyńska D, Wocław A, Gruszczyńska K, Nawrat Z. Multi-Label Classification of Chest X-ray Abnormalities Using Transfer Learning Techniques. J Pers Med. 2023 Sep 22;13(10):1426. doi: 10.3390/jpm13101426. PMID: 37888037; PMCID: PMC10607847.